🧪 BÀI THỰC HÀNH PROXY PATTERN

✅ Backend: ASP.NET Core Web API
✅ Frontend: React (gọi API)
✅ Áp dụng Proxy Pattern đúng chuẩn GoF
✅ Minh họa: **Kiểm soát truy cập + caching + logging**

📌 BÀI TOÁN

Hệ thống E-commerce có API lấy thông tin sản phẩm.

👉 Nhưng:

- Không phải user nào cũng được truy cập
- Cần cache để tăng performance
- Cần log request

➡️ Giải pháp: **Proxy Pattern**

📦 KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
        ↓
GET /api/products/{id}
        ↓
ProductController
        ↓
ProductProxy (Proxy)
        ↓
RealProductService
```

💻 BACKEND – ASP.NET Core 8 Web API

1️⃣ Tạo project

```
dotnet new webapi -n ProxyPatternDemo
cd ProxyPatternDemo
```

🗂 Cấu trúc thư mục

```
ProxyPatternDemo
  ├── Controllers
  │     └── ProductsController.cs
  ├── Services
  │     ├── IProductService.cs
  │     ├── RealProductService.cs
  │     └── ProductProxy.cs
  ├── Models
  │     └── Product.cs
  └── Program.cs
```

---

1. SUBJECT (INTERFACE)

Services/IProductService.cs

```
namespace ProxyPatternDemo.Services;

using ProxyPatternDemo.Models;

public interface IProductService
{
    Task<Product> GetProduct(int id);
}
```

---

2. REAL SUBJECT

Services/RealProductService.cs

```
using ProxyPatternDemo.Models;

namespace ProxyPatternDemo.Services;

public class RealProductService : IProductService
{
    public async Task<Product> GetProduct(int id)
    {
        // Giả lập call DB
        await Task.Delay(1000);

        return new Product
        {
            Id = id,
            Name = $"Product {id}",
            Price = 100 + id
        };
```

```
    }
}
```

🧩 3. PROXY (CORE PATTERN)

Services/ProductProxy.cs

```
using ProxyPatternDemo.Models;

namespace ProxyPatternDemo.Services;

public class ProductProxy : IProductService
{
    private readonly RealProductService _realService;
    private static Dictionary<int, Product> _cache =
new();

    public ProductProxy()
    {
        _realService = new RealProductService();
    }

    public async Task<Product> GetProduct(int id)
    {
        Console.WriteLine("Proxy: Checking access...");

        // 🔐 Access Control
        if (id <= 0)
            throw new Exception("Unauthorized access!");

        // ⚡ Caching
        if (_cache.ContainsKey(id))
        {
            Console.WriteLine("Proxy: Return from
cache");
            return _cache[id];
        }

        Console.WriteLine("Proxy: Calling real
service...");

        var product = await _realService.GetProduct(id);

        _cache[id] = product;
```

```
        return product;
    }
}
```

---

4. MODEL

Models/Product.cs

```
namespace ProxyPatternDemo.Models;

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

---

5. CONTROLLER

Controllers/ProductsController.cs

```
using Microsoft.AspNetCore.Mvc;
using ProxyPatternDemo.Services;

namespace ProxyPatternDemo.Controllers;

[ApiController]
[Route("api/products")]
public class ProductsController : ControllerBase
{
    private readonly IProductService _productService;

    public ProductsController(IProductService
productService)
    {
        _productService = productService;
    }
    [HttpGet("{id}")]
    public async Task<IActionResult> Get(int id)
    {
        var product = await
_productService.GetProduct(id);
        return Ok(product);
    }
}
```

---

6. ĐĂNG KÝ DI

Program.cs

```
using ProxyPatternDemo.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Proxy được inject thay vì RealService
builder.Services.AddSingleton<IProductService,
ProductProxy>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                  .AllowAnyHeader()
                  .AllowAnyMethod();
        });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.UseCors("AllowReact");

app.MapControllers();
app.Run();
```

▶ CHẠY BACKEND

```
dotnet run
```

Swagger:

```
https://localhost:xxxx/swagger
```

Test API:

```
GET /api/products/1
```

💻 FRONTEND – REACT

Tạo project

```
npx create-react-app proxy-ui
cd proxy-ui
npm install axios
```

---

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [id, setId] = useState(1);
  const [result, setResult] = useState(null);

  const getProduct = async () => {
    const res = await axios.get(
      `https://localhost:xxxx/api/products/${id}`
    );

    setResult(res.data);
  };

  return (
    <div style={{ padding: 40 }}>
      <h2>Product Viewer</h2>

      <input
        type="number"
        value={id}
        onChange={(e) => setId(e.target.value)}
      />
```

```
      <br /><br />

      <button onClick={getProduct}>Get Product</button>

      {result && (
        <div>
          <h3>{result.name}</h3>
          <p>Price: {result.price}</p>
        </div>
      )}
    </div>
  );
}

export default App;
```

▶ CHẠY FRONTEND

```
npm start
```

🔥 LUỒNG XỬ LÝ (PROXY)

Khi gọi API lần đầu

```
GET /api/products/1
```

Flow:

```
Controller
→ ProductProxy
   → check access
   → cache miss
   → gọi RealProductService
   → lưu cache
→ trả kết quả
```

Khi gọi lần 2 (cache hit)

```
Proxy: Return from cache
```

→ Không gọi DB nữa → nhanh hơn

🎯 TỔNG KẾT PROXY PATTERN

Thành phần	Vai trò
IProductService	Subject
RealProductService	Real Subject
ProductProxy	Proxy
Controller	Client

🚀 Ý NGHĨA THỰC TẾ

Proxy dùng khi:

- ✅ Bảo mật (Auth / Permission)
- ✅ Caching (Redis, Memory)
- ✅ Logging / Monitoring
- ✅ Lazy loading
- ✅ Remote proxy (Microservices)

🧠 BÀI TẬP MỞ RỘNG

1. Thêm JWT → Proxy kiểm tra token
2. Thay cache bằng Redis
3. Log request vào database
4. Giới hạn rate limit (API Gateway style)
5. Kết hợp Proxy + Factory